COL 774 – Machine Learning – Assignment 1

Due Date:
7:00 pm on Tuesday, 18th August, 2026 - Part 1 (Linear Regression)
7:00 pm on Sunday, 6th September, 2024 - Part 2 (Logistic Regression)

Max Marks : 100

Notes:

- **Copyright Claim:** This is a property of Prof. Rahul Garg Indian Institute of Technology Delhi, any replication of this without explicit permissions on any websites/sources will be considered violation to the copyright.
- All the announcements related to the assignment will be made on Piazza. The access code is col7747341. You are strongly advised to have a look at the relevant Piazza post regularly.
- This assignment has two parts: 1.1 – Linear Regression and 1.2 – Logistic Regression. They will be released separately.
- You are advised to use vector operations using numpy (wherever possible) for best performance.
- You should only use Python 3.
- Your assignments will be auto-graded on our servers, make sure you test your programs with the provided evaluation scripts before submitting. We will use your code to train the model on a different training data set and predict on a different test set as well.
- Input/output format, submission format and other details are included. Your programs should be modular enough to accept specified parameters. Any correction required after assignment submission deadline will incur 10% penalty.
- You may submit the assignment **individually or in groups of 2**. No additional resource or library is allowed except for the ones specifically mentioned in the assignment statement. If you still wish to use some other library, please consult on piazza. If you choose to use any external resource or copy any part of this assignment, you will be awarded D or F grade or **DISCO**.
- *Graceful degradation:* For each late day from the deadline, there will be a penalty of 7%. So, if you submit, say, 4 days after the deadline, you will be graded out of 72% of the weightage of the assignment. Any submission made after 7 days would see a fixed penalty of 50%.
- For doubts, use Piazza. Send an email to Ansh Agrawal and Mayank Shukla (both) for doubts that may disclose implementation details.

History:

- 7th Aug: Part 1 problem statement is released. Dataset link is here and Evaluation scripts will be added soon. Dataset is accessible over Institute Wifi and VPN.
- Part D dataset has been added and please check the evaluation scripts here.For queries please ask on piazza.
- 28th August: The dataset for Part 2 is at the link and data for part(c) will be added shortly .
- The dataset for Part C is here. along with the Kaggle Leaderboard.

Part 1: Linear Regression

In this problem, we will use physiological recordings from the **PhysioCGM dataset**. The dataset contains multimodal recordings collected from participants with Type 1 diabetes using an Empatica E4 wristband, a Zephyr BioHarness chest strap and a Dexcom G6 Continuous Glucose Monitor (CGM).
This problem contains two prediction tasks. In Parts (a)–(c), you will predict the time-aligned heart-rate value exported by the Empatica E4. Heart rate is the number of cardiac beats per minute and is expressed in beats per minute (bpm). The E4 derives heart rate from its photoplethysmography signal by detecting successive pulse waveforms and estimating the time intervals between them. It reports one heart-rate estimate per second, with each estimate computed using a longer signal span.
For a heart-rate target at time t, every example in Parts (a)–(c) contains:

- 640 Blood Volume Pulse (BVP) samples recorded at 64 Hz during the preceding ten-second interval $[t-10, t)$;
- 320 samples from each of the three accelerometer axes recorded at 32 Hz during the ten-second interval $[t-10, t)$, giving $320 \times 3 = 960$ acceleration values; and
- 40 Electrodermal Activity (EDA) samples recorded at 4 Hz during the ten-second interval $[t-10, t)$.

BVP is the relative optical waveform obtained using the Empatica E4 photoplethysmography sensor and is represented in relative device units. The accelerometer records motion along three orthogonal axes. EDA measures electrical conductance at the skin surface and is expressed in microsiemens (μS). Thus, every heart-rate example contains:

$$m = 640 + (320 \times 3) + 40 = 1640 \tag{1}$$

input values and one heart-rate target in bpm. The ten-second interval is divided into ten consecutive one-second blocks. Block $s = 0$ is the oldest block, corresponding to $[t-10, t-9)$, and block $s = 9$ is the most recent block, corresponding to $[t-1, t)$.
In Part (d), you will estimate CGM glucose using physiological recordings from the Empatica E4 and Zephyr BioHarness during the five-minute interval preceding each CGM measurement. The target is the corresponding interstitial glucose concentration, expressed in milligrams per decilitre (mg/dL).

Evaluation Metrics
The evaluation scripts will report Normalized Mean Absolute Error (NMAE) and Normalized Mean Squared Error (NMSE). For an evaluation set containing q examples, define the mean target value as:

$$\overline{y} = \frac{1}{q} \sum_{i=1}^{q} y^{(i)}. \tag{2}$$

The Normalized Mean Absolute Error is:

$$NMAE = \frac{\sum_{i=1}^{q} \left| y^{(i)} - \hat{y}^{(i)} \right|}{\sum_{i=1}^{q} \left| y^{(i)} - \overline{y} \right|}. \tag{3}$$

The Normalized Mean Squared Error is:

$$NMSE = \frac{\sum_{i=1}^{q} \left(y^{(i)} - \hat{y}^{(i)} \right)^2}{\sum_{i=1}^{q} \left(y^{(i)} - \overline{y} \right)^2}. \tag{4}$$

Both metrics are dimensionless, and lower values indicate better performance. A value of zero represents perfect prediction. A value of one corresponds to the respective error obtained by predicting the mean target value for every example. You may assume that the denominators are non-zero for all evaluation sets. Your programs will be evaluated automatically on unseen data. Do not assume a fixed number of training examples, test examples or participants. The order of the test examples must be preserved. Input and output paths must be read from the command-line arguments and must not be hard-coded.

1. **Part (a): Closed-Form Linear Regression – 10 points**

In this part, you will implement ordinary least-squares linear regression using its closed-form solution. You will be provided with `train.csv`, containing the input features followed by the target heart rate, and `test.csv`, containing the same input features without the target.

The columns are grouped by one-second block. For each $s \in \{0, 1, \ldots, 9\}$, the columns appear in the following order:

```
acc_x_s{s}_0,...,acc_x_s{s}_31,
acc_y_s{s}_0,...,acc_y_s{s}_31,
acc_z_s{s}_0,...,acc_z_s{s}_31,
bvp_s{s}_0,...,bvp_s{s}_63,
eda_s{s}_0,...,eda_s{s}_3
```

The complete CSV contains the block for $s = 0$, followed by the blocks for $s = 1, \ldots, 9$. The training file contains the target column `hr` after all 1640 feature columns. The test file contains the same feature columns but does not contain `hr`.

Thus, each one-second block contains:

$$(32 \times 3) + 64 + 4 = 164 \tag{5}$$

features, and the complete ten-second example contains:

$$m = 10 \times 164 = 1640 \tag{6}$$

features.

The feature columns must be used in exactly the order in which they appear in the supplied files.

Let the training dataset be:

$$D = \left\{ \left(x^{(i)}, y^{(i)} \right) \right\}_{i=1}^{n},$$

where $x^{(i)} \in \mathbb{R}^m$ is the feature vector of example i, $m = 1640$, and $y^{(i)} \in \mathbb{R}$ is the corresponding heart rate.

The ordinary least-squares optimization problem is:

$$\min_{w,b} \sum_{i=1}^{n} \left(w^T x^{(i)} + b - y^{(i)} \right)^2, \tag{7}$$

where $w \in \mathbb{R}^m$ is the feature-weight vector and $b \in \mathbb{R}$ is the bias.

Define the augmented feature vector:

$$\widetilde{x}^{(i)} = \begin{bmatrix} 1 \\ x^{(i)} \end{bmatrix} \in \mathbb{R}^{m+1}.$$

Construct the augmented design matrix $X \in \mathbb{R}^{n \times (m+1)}$, target vector $Y \in \mathbb{R}^n$ and parameter vector $W \in \mathbb{R}^{m+1}$ as:

$$X = \begin{bmatrix} (\widetilde{x}^{(1)})^T \\ (\widetilde{x}^{(2)})^T \\ \vdots \\ (\widetilde{x}^{(n)})^T \end{bmatrix}, \qquad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}, \qquad W = \begin{bmatrix} b \\ w \end{bmatrix}. \tag{8}$$

The objective can therefore be written as:

$$\min_{W} \|XW - Y\|_2^2.$$

The closed-form solution is:

$$W = \left(X^T X \right)^{-1} X^T Y. \tag{9}$$

The first component of W represents the intercept. You must use `numpy.linalg.inv` to calculate the inverse. The matrix to be inverted will be invertible, and you do not need to handle the singular case.

You may use NumPy for computation and pandas for reading the data. You may not use `numpy.linalg.pinv`, scikit-learn regression models or any other ready-made implementation of linear regression.

Your file must be named `part_a.py` and will be called as:

```
python3 part_a.py train.csv test.csv predictions.txt weights.txt
```

The file `predictions.txt` must contain one predicted heart-rate value per line, preserving the order of `test.csv`. The file `weights.txt` must contain one model parameter per line, with the intercept first, followed by the m feature coefficients in the same order as the input columns.

The output in **weights.txt** and **predictions.txt** must match the deterministic reference outputs within the numerical tolerance used by the evaluator. **Do not shuffle rows, reorder columns, standardize the features, remove examples or create additional features in this part.**

2. **Part (b): Ridge Regression with Five-Fold Cross-Validation – 15 points**

In this part, you will implement ridge regression and use five-fold cross-validation to select the regularization parameter. You will use the same feature representation as in Part (a). Feature creation, feature selection and feature standardization are not allowed in this part.

In addition to `train.csv` and `test.csv`, you will be provided with:

- `folds.txt`, containing the ending index for each fold, please refer to reference folds.txt ; and
- `regularization.txt`, containing the candidate values of λ, one value per line.

The ridge-regression optimization problem is:

$$\min_{w,b} \frac{1}{2} \sum_{i=1}^{n} \left(w^T x^{(i)} + b - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{m} w_j^2, \tag{10}$$

where $\lambda \geq 0$ is the regularization parameter. The intercept must not be regularized.

Using the augmented matrix X, target vector Y and parameter vector W defined in Part (a), define:

$$R = \text{diag}(0, 1, 1, \ldots, 1), \tag{11}$$

where $R \in \mathbb{R}^{(m+1)\times(m+1)}$. The zero in the first diagonal entry ensures that the intercept is not regularized.

The closed-form ridge-regression solution is:

$$W_\lambda = \left(X^T X + \lambda R \right)^{-1} X^T Y. \tag{12}$$

You must use `numpy.linalg.inv` to calculate the inverse. You may not use a ready-made implementation of ridge regression.

Now we will use K-fold cross-validation to select the best λ. Let: $\mathcal{I} = \{1, 2, \ldots, n\}$ denote the indices of all training examples. The file `folds.txt` partitions these indices into five mutually disjoint sets:

$$\mathcal{F}_0, \mathcal{F}_1, \mathcal{F}_2, \mathcal{F}_3, \mathcal{F}_4,$$

such that:

$$\mathcal{F}_j \cap \mathcal{F}_k = \emptyset \quad \text{for all } j \neq k, \qquad \bigcup_{k=0}^{4} \mathcal{F}_k = \mathcal{I}. \tag{13}$$

For fold k, use the examples indexed by $\mathcal{I} \setminus \mathcal{F}_k$ for training and the examples indexed by $\mathcal{F}_k$ for validation. Let $\hat{y}_{\lambda,k}^{(i)}$ denote the prediction for validation example $i \in \mathcal{F}_k$ obtained using the model trained outside fold k with regularization parameter λ.

Define the mean target value in validation fold k as:

$$\overline{y}_k = \frac{1}{|\mathcal{F}_k|} \sum_{i \in \mathcal{F}_k} y^{(i)}. \tag{14}$$

The fold-wise Normalized Mean Squared Error is:

$$NMSE_k(\lambda) = \frac{\sum_{i \in \mathcal{F}_k} \left(y^{(i)} - \hat{y}_{\lambda,k}^{(i)} \right)^2}{\sum_{i \in \mathcal{F}_k} \left(y^{(i)} - \overline{y}_k \right)^2}. \tag{15}$$

The overall five-fold cross-validation error is:

$$CV_{\text{NMSE}}(\lambda) = \frac{1}{5}\sum_{k=0}^{4} NMSE_k(\lambda). \tag{16}$$

Select the value of λ having the smallest $CV_{\text{NMSE}}(\lambda)$. If multiple values produce the same cross-validation error, select the first such value in the order in which it appears in `regularization.txt`. After selecting the best value of λ, retrain ridge regression on the complete training dataset and generate predictions for `test.csv`.

You may use NumPy for computation and pandas for reading data. You may not use `numpy.linalg.pinv`, scikit-learn regression models, scikit-learn cross-validation utilities or any other ready-made implementation of ridge regression.

Your file must be named `part_b.py` and will be called as:

```
python3 part_b.py train.csv test.csv folds.txt regularization.txt
predictions.txt weights.txt bestlambda.txt crossvalidation_errors.txt
```

The entire process of must complete within **30 minutes** and use at most **24 GB** of memory on Kaggle with T4 GPU enabled.

The files **predictions.txt** and **weights.txt** must follow the same formats as in Part (a). The file `bestlambda.txt` must contain only the selected value of λ.

The file `crossvalidation_errors.txt` must contain one comma-separated pair for every candidate value, preserving the order in `regularization.txt`:

```
0.0,0.032819
0.1,0.031940
1.0,0.029182
10.0,0.028751
100.0,0.034291
```

Each line must contain the candidate value of λ, followed by its five-fold CV_{NMSE} score.

3. **Part (c): Feature Engineering for Heart-Rate Prediction – 10 points**

This part will be graded in a competitive manner.

In this part, you will improve heart-rate prediction by creating informative features from the same physiological input used in Parts (a) and (b): ten seconds of BVP ,accelerometer and EDA data.

Possible BVP features include statistical summaries, consecutive differences, slopes, signal energy, quantiles, polynomial terms and autocorrelation values. Accelerometer features may include axis-wise statistics, squared acceleration magnitude, movement variability and interactions between axes. EDA features may include its mean, range, slope and variability. These examples are only indicative, and you are encouraged to create other meaningful features. [1] [2] [3]

Library Rules

Feature creation must be implemented by you. For feature creation, you may use:

- Python's standard library;
- NumPy, except the `numpy.fft` module; and
- pandas.

You may use scikit-learn only for:

- scaling and preprocessing of your manually created features;
- cross-validation and hyperparameter selection;
- feature selection; and
- fitting a permitted linear-regression model.

You may not use scikit-learn, SciPy or any other library to automatically generate time-series or physiological features. Ready-made feature-extraction libraries such as `tsfresh`, `NeuroKit2`, `HeartPy`, `librosa` and equivalent packages are not permitted in this part. PCA and other automatic dimensionality-reduction methods are also not permitted.

The final prediction model must be linear in the created feature representation. Permitted final models include ordinary linear regression, ridge regression, Lasso , Elastic net or other linear models. Decision trees, random forests, boosting methods, nearest-neighbour regression, nonlinear support-vector regression, neural networks and other nonlinear predictive models are not permitted.

Any learned preprocessing or feature-selection operation must be fitted using the training data only. The entire process of data reading, feature creation, model training and test prediction must complete within ~~**10**~~ **30 minutes** and use at most **24 GB** of memory on Kaggle with T4 GPU enabled.

Your submission file must be named `part_c.py` and will be called as:

```
python3 part_c.py train.csv test.csv predictions.txt
```

Your program must write one predicted heart-rate value per line to the output path supplied as the third command-line argument. The prediction order must match the order of the examples in `test.csv`.

This part will be evaluated using **hidden test set**. We will announce a threshold for minimum qualifying performance on piazza. Let the announced qualifying threshold be T_{HR}.

- Submissions with $NMAE > T_{\mathrm{HR}}$ will receive 0 marks.
- Every submission satisfying $NMAE \leq T_{\mathrm{HR}}$ will receive at least 35% of the marks allocated to this part.
- Among the qualifying submissions, the top 25% will receive 100% of the marks.
- The next 25% will receive 75% of the marks.
- The remaining qualifying submissions will receive between 35% and 75% linearly scaled, based on their relative hidden-test performance.

The evaluation script will report NMSE and NMAE on the public test data while the private test data will be used for final grading.

4. **Part (d): CGM Glucose Estimation – 15 points**

This part will be graded in a competitive manner.

Continuous and reliable glucose monitoring is important for diabetes management. Conventional blood-glucose measurements require either a blood sample or a sensor inserted under the skin. Consequently, there is considerable interest in estimating glucose non-invasively using physiological signals collected by wearable devices. Photoplethysmography has previously been investigated for non-invasive glucose estimation. For example, Monte-Moreno (2011) studied glucose estimation using PPG-derived physiological features and machine-learning models. In this part, you will investigate whether physiological signals recorded by the Empatica E4 and Zephyr BioHarness contain information useful for estimating CGM glucose.

Effect of Data Splitting

Physiological datasets often contain many signal windows from a relatively small number of participants. Measurements from the same participant are not independent: they may share participant-specific anatomy, PPG morphology, temporally persistent glucose behaviour. If individual windows are randomly shuffled into training and test sets, windows from the same participant and nearby time points may appear in both partitions. A model may then exploit participant-specific or temporally persistent information and obtain an overly optimistic evaluation score.

A similar issue has been demonstrated for repeated EEG segments in studies of subject-wise data splitting. Although that work considered EEG, the same concern applies to PPG and wearable-sensor data.

You will therefore compare random within-subject, temporal within-subject and cross-subject evaluation protocols and try to train models that work in all these settings.

Task

In this part, you will estimate CGM glucose concentration from five-minute windows of physiological signals recorded using the Empatica E4 and Zephyr BioHarness. You will also study how random within-subject, temporal within-subject and cross-subject data splits affect the measured generalization performance of your model.

For every CGM measurement recorded at time t_i,the corresponding input contains physiological signals recorded during:

$$[t_i - 5 \text{ minutes}, t_i). \tag{17}$$

Let $\mathcal{X}^{(i)}$ denote the multimodal physiological window of example i. The corresponding target $y^{(i)}$ is the interstitial glucose concentration recorded by the CGM device at time t_i, expressed in mg/dL.

Each Part (d) input path is a directory containing one `.npz` file per participant. The program must read only the `.npz` files directly inside the supplied directory, sorted in lexicographic order of filename. The complete dataset is formed by concatenating files in this order and preserving the row order within every file. Training directories contain the `glucose` array. Test directories contain the same fields except `glucose`. Students must not remove, reorder or duplicate test examples. The output feature matrix must follow the concatenated order defined above.

For a file containing N windows, the arrays are:

```
e4_bvp           : N x 19200 array       (64 Hz x 300 s)
e4_hr            : N x 300 array         (1 Hz x 300 s)
e4_eda           : N x 1200 array        (4 Hz x 300 s)
e4_temp          : N x 1200 array        (4 Hz x 300 s)
e4_acc           : N x 9600 x 3 array    (32 Hz x 300 s)
zephyr_ecg       : N x 75000 array       (250 Hz x 300 s)
zephyr_acc       : N x 30000 x 3 array   (100 Hz x 300 s)
zephyr_breathing : N x 7500 array        (25 Hz x 300 s)
subject_id       : N-element array
timestamp        : N-element array
glucose          : N-element target array, in mg/dL (training files only)
```

For every time-series array, samples are ordered chronologically from oldest to newest. The input window for a CGM reading at time ti is [ti 5 minutes, ti). Test files contain every field above except `glucose`. The required example order is the order of input paths on the command line, followed by the existing row order within each '.npz' file. Students must not remove, reorder, or duplicate test examples. The output feature matrix must follow this concatenated order. ~~The field `sample_id` uniquely identifies each example and determines the required output order.~~ The fields `subject_id` and `timestamp` may be used for validation splitting, temporal ordering and visualization. They may not be included as predictive features, used for target encoding, used to retrieve hidden glucose targets or used to select a different prediction model for each test participant.

Missing physiological values will be represented by `numpy.nan`. You may impute or otherwise handle missing values, but all imputation parameters must be learned using training data only. You may remove training examples if required, but you must not remove, reorder or duplicate any test example. The final submitted feature matrix must contain only finite values.

Feature Creation, Selection and Model Restrictions

You may use any subset of the supplied physiological modalities and may create any physiologically meaningful features. You may use any feature-creation, feature-selection or preprocessing method available in the supplied evaluation environment. NumPy, pandas, SciPy, scikit-learn, PyWavelets and NeuroKit2 may be used. Additional libraries may be used only if they are explicitly included in the supplied `requirements.txt` file. Please ask on Piazza for any other library required. **The final feature matrix must contain fewer than 500 features.**

The final prediction model must be linear in the final generated feature representation. If $z^{(i)} \in \mathbb{R}^m$ is the final feature vector, the prediction must have the form:

$$\hat{y}^{(i)} = w_0 + w^T z^{(i)}. \tag{18}$$

Permitted final models include ordinary linear regression, ridge regression, Lasso, elastic net, and other models whose final prediction can be represented using one intercept and one coefficient vector. Decision trees, random forests, gradient-boosting models, nearest-neighbour regression, nonlinear support-vector regression, neural networks and other final models whose predictions cannot be reproduced as $w_0 + Zw$ are not permitted.

All preprocessing, imputation, scaling, filtering, feature selection, dimensionality reduction, hyperparameter tuning and model-selection decisions must use only the supplied training data. The test data must not be used to fit or refit any transformation.No internet access will be available during evaluation. Your program must not download external datasets, pretrained models, packages or other resources.

You may use AI tools while developing this part. However, you are responsible for the submitted code and must understand the complete feature-generation, model-selection and prediction pipeline. Your understanding will be tested during the viva.

Evaluation Protocols

(i) **Random within-subject split (`d1`) – 1 point:**
Training and test examples are randomly selected from shuffled windows belonging to the same participant or set of participants. The same participant and temporally related windows may occur in both sets. This protocol is intentionally included to demonstrate how a non-independent split can produce an optimistic estimate of performance.

(ii) **Temporal within-subject split (`d2`) – 7 points:**
Earlier physiological windows from a participant are used for training, while later windows from the same participant are used for testing. This evaluates prediction on future observations from participants already observed during training.

(iii) **Cross-subject split (`d3`) – 7 points:**
The training and test files contain disjoint sets of participants. No test participant is present in the corresponding training file. This evaluates generalization to previously unseen participants.

A separate training and test folder will be supplied for each protocol. Any local validation strategy must use only the corresponding training folder and should preserve the intended splitting protocol.

Program Interface

Your submission file must be named `part_d.py`. It must support two modes:

(i) `train`: create training features, fit all preprocessing operations, select the final features, train the final linear model and save the complete model state;

(ii) `feature_engineering`: load the saved state, apply the identical feature transformation to the supplied test data and save the final test-feature matrix.

The training commands will be (Note the change in case of train folder path which will have subject-wise train files):

```
python3 part_d.py train d1 train_d1/ model_d1.pkl
python3 part_d.py train d2 train_d2/ model_d2.pkl
python3 part_d.py train d3 train_d3/ model_d3.pkl
```

The complete trained state must be stored in the specified pickle file. No separate weight file should be generated.

The pickle file must contain a dictionary with the following mandatory keys:

```
{
    "format_version": 1,
    "protocol": "d1",
    "intercept": <floating-point scalar>,
    "coef": <one-dimensional NumPy array>,
    "feature_names": <list of strings>,
    "preprocessing_state": <pickle-compatible object>
}
```

The value of `protocol` must match the protocol argument. The field `intercept` must contain the final intercept w_0, and `coef` must contain the final coefficient vector $w \in \mathbb{R}^m$.

The list `feature_names` must contain exactly m strings in the same order as the coefficients:

$$\text{len}(\texttt{coef}) = \text{len}(\texttt{feature\_names}) = m. \tag{19}$$

The field `preprocessing_state` may contain fitted scalers, selected feature indices, dimensionality-reduction parameters, filters or other training-derived objects required to transform test data.

The pickle must not contain the complete raw training dataset, the complete target vector, hidden test information or hard-coded test predictions.

The saved intercept and coefficients must directly produce predictions in mg/dL. If the target was transformed during training, the coefficients and intercept must be converted back to the original glucose scale before saving.

The feature-engineering commands will be:

```
python3 part_d.py feature_engineering d1 test_d1/ model_d1.pkl features_d1.npy
python3 part_d.py feature_engineering d2 test_d2/ model_d2.pkl features_d2.npy
python3 part_d.py feature_engineering d3 test_d3/ model_d3.pkl features_d3.npy
```

The output file must contain a two-dimensional NumPy array:

$$Z_{\text{test}} \in \mathbb{R}^{n_{\text{test}} \times m}. \tag{20}$$

The feature matrix must not contain a bias column. All values must be finite, and the row order must match the original test-example order. The columns must correspond exactly to the entries of `feature_names` and `coef`. The evaluator will calculate $\widehat{Y}_{\text{test}} = w_0 \mathbf{1}_{n_{\text{test}}} + Z_{\text{test}} w$.

No additional clipping, inverse transformation or prediction post-processing will be performed by the evaluator.

`d1`,`d2` and `d3` will be graded competitively using hidden-test NMAE. NMSE will also be reported. The exact relative-marking rules will be announced with the evaluation scripts. The complete training and feature-engineering process for every protocol must finish within ~~**20**~~ **30 minutes** and use at most **24 GB** of memory on a Kaggle notebook with T4 GPU enabled.

5. **Part (e): Open Research Challenge – Up to 200 Bonus Points**

This is an open optional problem based on the CGM glucose-estimation task in Part (d). Now you may use any model architecture, feature representation, loss function or model-selection method available in the supplied evaluation environment. The requirement that the final model be linear does not apply to this part.

You may use linear models, tree-based methods, deep neural networks or other suitable approaches. All experiments must respect the same evaluation protocols described in Part (d). Test targets, publicly reconstructed hidden targets and external labelled copies of the evaluation data must not be used.

The program must follow the same two-stage train-and-evaluate pattern as Part (d). Your file must be named `part_e.py` and must support the following commands:

```
python3 part_e.py train d1 train_d1/ model_e_d1.pkl
python3 part_e.py predict d1 test_d1/ model_e_d1.pkl predictions_e_d1.txt

python3 part_e.py train d2 train_d2/ model_e_d2.pkl
python3 part_e.py predict d2 test_d2/ model_e_d2.pkl predictions_e_d2.txt

python3 part_e.py train d3 train_d3/ model_e_d3.pkl
python3 part_e.py predict d3 test_d3/ model_e_d3.pkl predictions_e_d3.txt
```

The prediction files must contain one finite glucose prediction per line, preserve the test-example order and directly report predictions in mg/dL.

To receive the 10 bonus marks, both of the following conditions must be satisfied:

(i) the method must achieve instructor-verified performance exceeding the accepted state-of-the-art benchmark . **Details will be shared

(ii) the work must be developed into a complete, reproducible and submission-ready research paper.

Your code, experiments and manuscript will be fully evaluated by the instructors. Merely exceeding an internal course baseline, or preparing a manuscript without sufficient experimental evidence, will not qualify for the bonus marks.

The deadline for Part (e) is **17 November 2026**. Part (e) is optional and carries up to 10 bonus marks in addition to the regular assignment marks. Detailed submission instructions for the research code, manuscript and supplementary material will be announced separately.

Report

Submit a concise report of at most **three pages**. All plot axes must be clearly labelled and must include the appropriate units.

1. **Created features:**

   Briefly describe the final features created for Parts (c) and (d), the physiological modality from which each feature family was obtained, and the final linear model used.

2. **Normalized acceleration:**

   For accelerometer samples $(a_x(t), a_y(t), a_z(t))$, calculate the squared acceleration magnitude:

   $$a_{\text{sq}}(t) = a_x(t)^2 + a_y(t)^2 + a_z(t)^2. \tag{21}$$

   Normalize it using its temporal mean:

   $$\widetilde{a}_{\text{sq}}(t) = \frac{a_{\text{sq}}(t)}{\frac{1}{T}\sum_{\tau=1}^{T} a_{\text{sq}}(\tau)}. \tag{22}$$

   Plot $\widetilde{a}_{\text{sq}}(t)$ for the representative windows described below.

   (i) **Representative Part (a) window:**

   Plot one representative heart-rate example showing:

   - the ten-second BVP waveform;
   - the normalized squared acceleration magnitude for the 10 second interval;
   - the 40 EDA samples ; and
   - the corresponding heart-rate target.

   (ii) **Representative Part (d) window:**

   Plot one representative five-minute glucose-estimation example showing the available:

   - BVP waveform;
   - normalized squared acceleration magnitude;
   - EDA signal;
   - ECG signal;
   - heart-rate signal; and
   - corresponding CGM glucose target.

3. **Top five features:**

   For both Parts (c) and (d), report the five most important final features. Since the final models are linear, feature importance should be calculated using the absolute standardized regression coefficient. Standardize the final features before comparing coefficient magnitudes.

   For every reported feature, provide:

   - its name;
   - the signal modality from which it was created;
   - its absolute standardized coefficient; and
   - a one-line explanation of why it may be relevant.

4. **Median baseline comparison:**

   For Parts (c) and (d), compare your model with a baseline that predicts the median training target for every validation example:

   $$\hat{y}^{(i)}_{\text{median}} = \text{median}\left(y_{\text{train}}\right). \tag{23}$$

   Report NMAE and NMSE for both your model and the median baseline using the same local validation examples. For Part (d), report the comparison separately for the random within-subject, temporal within-subject and cross-subject validation settings.

Submit `report.pdf` along with your code on Moodle.

What Have You Been Given?

- `train.csv` and `test.csv` for Parts (a)–(c);
- `folds.txt` and `regularization.txt` for Part (b);
- development target files for local evaluation of Parts (c) and (d);
- sample reference weights and predictions for Part (a);
- sample reference weights, predictions, cross-validation errors and selected regularization parameter for Part (b);
- protocol-specific `train_d1/`, `test_d1/`, `train_d2/`, `test_d2/`, `train_d3/` and `test_d3/` directories, each containing participant-specific `.npz` files for Part (d);
- evaluation scripts for all four parts; and
- a `requirements.txt` file describing the final evaluation environment.

Detailed Submission Instructions

Moodle submission:
Submit a single ZIP archive named using your entry number:

`2024CSXXXXX.zip`

For a group submission, use both entry numbers in lexicographic order:

`2024CSXXXXX_2024CSYYYYY.zip`

The ZIP archive must contain one folder having exactly the same name as the archive without the `.zip` extension. The folder must contain exactly:

```
part_a.py
part_b.py
part_c.py
part_d.py
report.pdf
```

Do not include datasets, generated predictions, weights, pickle files, NumPy feature files, cache directories, virtual environments, installed packages or library source files.

Command-line requirements:
Your programs must run using the following interfaces:

```
python3 part_a.py train.csv test.csv predictions.txt weights.txt

python3 part_b.py train.csv test.csv folds.txt regularization.txt
predictions.txt weights.txt bestlambda.txt crossvalidation_errors.txt

python3 part_c.py train.csv test.csv predictions.txt

python3 part_d.py train d1 train_d1/ model_d1.pkl
python3 part_d.py feature_engineering d1 test_d1/ model_d1.pkl features_d1.npy
```

The corresponding Part (d) commands must also work for protocols `d2` and `d3`.
All input and output paths must be read from the command-line arguments. No file name, directory, participant identifier, number of examples or output path may be hard-coded. Do not assume that the program will be executed from the directory containing the input files. Your program must create every output at the exact path supplied through the command line.
All output files must contain the correct number of finite numerical values and must preserve the original test-example order.

Evaluation intervention penalty:

A penalty equal to **10% of the marks allocated to the corresponding part** will be applied for every change required to your submission during evaluation. Examples include:

- renaming a submitted source file;
- changing a hard-coded input or output path;
- modifying the order of command-line arguments;
- manually creating a missing output file;
- correcting an output-file format;
- editing submitted source code to make it execute;
- changing assumptions about the working directory; or
- removing files or dependencies that interfere with evaluation.

Multiple interventions will incur multiple 10% penalties. A program that cannot be executed through minor interventions may receive zero marks for that part.

Working with Part (d) `.npz` files

The files are compressed NumPy archives. Use `numpy.load` with `allow_pickle=False`:

```
with np.load("train_d1/c1s01.npz", allow_pickle=False) as data:
    print(data.files)
    bvp = data["e4_bvp"]
    glucose = data["glucose"]
```

Calling `numpy.load` is lightweight: it returns an `NpzFile` archive object. However, accessing an array such as `data["e4_acc"]` decompresses and loads that entire array into memory. Consequently, indexing after access does not avoid loading the full array:

```
with np.load(path, allow_pickle=False) as data:
    x = data["e4_acc"][100:200]  # the complete e4_acc array is loaded first
```

The recommended memory-efficient strategy is to process one participant file at a time. Load only the modalities required by your method, create a small feature matrix for that file, then discard the raw arrays before proceeding to the next file. Since the final feature matrix contains fewer than 500 features, the concatenated engineered features are substantially smaller than the raw signals.

```
all_features, all_targets = [], []

for path in sorted(Path(train_dir).glob("*.npz")):
    with np.load(path, allow_pickle=False) as data:
        bvp = data["e4_bvp"]
        eda = data["e4_eda"]
        y = data["glucose"]
        features = make_features(bvp, eda)

    all_features.append(features)
    all_targets.append(y)
```

Part 2: Logistic Regression

Introduction

Atrial fibrillation (AF) is the most common sustained cardiac arrhythmia and a major risk factor for stroke and heart failure. This assignment is inspired by the PhysioNet/Computing in Cardiology (CinC) 2017 Challenge, which asked entrants to classify short single-lead ECG recordings from an AliveCor handheld device as Normal rhythm, AF, or an Other rhythm.
You will build a **3-class classifier** for this task using **multinomial (softmax) logistic regression that you implement from scratch**. The three classes are:

$$y \in \{0, 1, 2\}, \qquad 0 = \text{Normal (N)}, \quad 1 = \text{AF (A)}, \quad 2 = \text{Other (O)}.$$

The assignment has three parts. Part (a) focuses on correctly implementing and optimising multinomial logistic regression using four gradient-descent variants, using a **prescribed feature representation** and **prescribed hyperparameters**. Part (b) studies the effect of class imbalance and asks you to compare static (class-weighting, at two strengths) and dynamic (focal loss) reweighting strategies, and to reason about which evaluation metrics actually reveal their effect. Part (c) is the open-ended, competitively graded component, where you design your own ECG features from the raw signal while keeping softmax logistic regression as the final classifier.

Data source and preprocessing

Every example is derived from a **30-second, single-lead ECG window**. Each recording contributes exactly one example, so it does not appear in more than one of train/validation/test.
For Parts (a) and (b), each window is represented by **78 hand-engineered features**, computed from the filtered, resampled (256 Hz) ECG signal. The feature columns fall into three groups:

- **RR-interval / heart-rate-variability features** (38 columns): statistics of the beat-to-beat intervals – mean, median, standard deviation, coefficient of variation, RMSSD, pNN50/pNN20, Poincaré SD1/SD2, sample entropy, autocorrelation, turning-point ratio, Hjorth parameters, and QRS-detection quality indicators.
- **Beat-morphology features** (16 columns): descriptors of a representative heartbeat template – energy, peak-to-peak amplitude, QRS width, P-wave/T-wave region energy, spectral coefficients of the beat shape, and beat-to-beat shape consistency.
- **Full-signal statistical/spectral features** (24 columns): moments of the raw window, zero-crossing rate, Hjorth parameters, dominant frequency, spectral entropy, sub-band power ratios, and multi-resolution filter-bank energies.

Please refer to the link for exact feature list and descriptions. You are not given the feature-extraction code, only the resulting feature values for Parts (a) and (b).
For Part (c), you are also given the **raw filtered signal** for each window (7680 samples at 256 Hz) and must design your own feature representation.

Evaluation metric

For Part (a), the primary reported metric is **macro-averaged Average Precision (Macro-AP)**: compute one-vs-rest AP for each of the three classes and average them. AP summarises the precision-recall curve obtained by ranking examples using the predicted class probability, and does not require choosing a decision threshold. It is preferred over plain accuracy here because the classes are imbalanced (Normal is the majority class); a classifier that ignores the minority classes can still achieve high accuracy while ranking AF and Other windows no better than chance.

$$\text{AP}_k = \sum_i (R_i - R_{i-1}) P_i, \qquad \text{MacroAP} = \frac{1}{3} \sum_{k \in \{N,A,O\}} \text{AP}_k.$$

We use the non-interpolated Average Precision definition implemented by `average_precision_score` For this three-class task, AP is computed one-vs-rest for each class using its predicted probability, and Macro-AP is their unweighted mean This corresponds to `average="macro"` in scikit-learn, see precision-recall metrics guide.
Part (b) requires you to report *additional* metrics and to reason about which of them actually reflects the benefit of an imbalance-handling strategy, please refer to the Report section at the end of this document.

Reproducibility. Unless a part explicitly permits otherwise, use the exact preprocessing, initialisation, update rule, iteration count, and random seed specified in that part. These requirements exist because submissions are autograded, an implementation that is correct but deviates from the specified protocol will not match the reference and will not receive marks.

Part (a): Gradient Descent Variants 20 points

You will implement multinomial logistic regression and train it with **four** different gradient-based optimisers. **All hyperparameters for this part are fixed and given to you below**.

Model

Let $x \in \mathbb{R}^{78}$ be a (standardised) feature vector, $W \in \mathbb{R}^{78\times 3}$ a weight matrix, and $b \in \mathbb{R}^3$ a bias vector. The model predicts

$$p(y = k \mid x) = \mathrm{softmax}(W^\top x + b)_k = \frac{\exp\big((W^\top x + b)_k\big)}{\sum_{j=0}^{2} \exp\big((W^\top x + b)_j\big)}.$$

Training minimises the mean cross-entropy loss (no regularisation term):

$$\mathcal{L}(W, b) = -\frac{1}{n}\sum_{i=1}^{n} \log p(y_i \mid x_i).$$

For numerical stability, the reference implementation subtracts the row-wise maximum logit before exponentiating (the standard softmax stabilisation trick, which makes every shifted logit ≤ 0) and additionally clips the shifted logits to $[-60, 0]$ before calling `exp`.

Data

`part_ab_train.csv` and `part_ab_val.csv` contain a `release_id` column, the 78 feature columns (see Introduction), and a `label` column (0/1/2). `part_ab_test_public.csv` contains `release_id` and the feature columns only (no label). `release_id` is a de-identified, per-row hash key (derived from the original PhysioNet record id) used to keep rows traceable and joinable across files (including the Kaggle leaderboard's `Id` column for Part (c)), it carries no predictive information and must **not** be used as a model input. Select the 78 named feature columns other than release_id. Standardise every feature to zero mean and unit variance using **training-set statistics only**, and apply the same transform to validation/test data. Do not recompute standardisation statistics on validation or test data.

Required optimisers

Let n be the number of training examples. Initialise $W^{(0)} = 0 \in \mathbb{R}^{78\times 3}$, $b^{(0)} = 0 \in \mathbb{R}^3$ in every case. Use `float64` arithmetic throughout. For a batch $B \subseteq \{1, \ldots, n\}$, write X_B, P_B, Y_B for the rows of the feature matrix, predicted probabilities, and one-hot labels restricted to B, and $|B|$ for the batch size – gradients below always divide by $|B|$ (the *current batch's* size), not by n (the full training set's size); the two coincide only for full-batch gradient descent, where $B = \{1, \ldots, n\}$.

Method 1: Full-batch gradient descent. At every epoch, compute the gradient over the *entire* training set ($B = \{1, \ldots, n\}$) and take one step:

$$g_W = \frac{1}{|B|} X_B^\top (P_B - Y_B), \qquad g_b = \frac{1}{|B|} \mathbf{1}^\top (P_B - Y_B),$$

$$W \leftarrow W - \eta_{\text{full}}\, g_W, \qquad b \leftarrow b - \eta_{\text{full}}\, g_b,$$

where $P_B \in \mathbb{R}^{n\times 3}$ is the matrix of predicted probabilities and $Y_B \in \mathbb{R}^{n\times 3}$ is the one-hot label matrix, over the full training set in this case. No shuffling occurs, there is exactly one update per epoch.

Method 2: Mini-batch gradient descent. At the **start of every epoch**, draw a fresh random permutation of $\{1, \ldots, n\}$ using `numpy.random.default_rng(seed)`, partition it into consecutive batches of size B_{mini}, and take one gradient step (as above, computed on each batch, with $|B| = B_{\text{mini}}$ except at the last, smaller batch) per batch.

Method 3: Stochastic gradient descent (SGD). Identical to mini-batch gradient descent with batch size 1.

Method 4: Mini-batch gradient descent with AdaGrad. As in Method 2, but maintain per-parameter accumulators $G_W \in \mathbb{R}^{78\times 3}$, $G_b \in \mathbb{R}^3$, initialised to zero, and update

$$G_W \leftarrow G_W + g_W \odot g_W, \qquad W \leftarrow W - \eta_{\text{ada}} \frac{g_W}{\sqrt{G_W} + \epsilon},$$

and analogously for b, G_b. All operations are element-wise.
After each **full epoch** (not each batch), record the full-training-set loss $\mathcal{L}(W, b)$.

Prescribed hyperparameters

Method	Parameter	Value
Full-batch GD	epochs T_{full}	`500`
	learning rate η_{full}	`0.3`
Mini-batch GD	epochs T_{mini}	`200`
	learning rate η_{mini}	`0.03`
	batch size B_{mini}	`32`
SGD	epochs T_{sgd}	`30`
	learning rate η_{sgd}	`0.001`
Mini-batch AdaGrad	epochs T_{ada}	`200`
	learning rate η_{ada}	`0.3`
	batch size B_{ada}	`32`
	stability constant ϵ	`1e-8`
All methods	shuffling seed	`774`

Run exactly the specified number of epochs for each method. Do not use early stopping, learning-rate schedules, momentum, or any optimiser other than the four specified above. Initialise rng = numpy.random.default_rng(774) once before training. At the start of each epoch, set order = rng.permutation(n). Process consecutive batches including the final, possibly smaller batch, and divide each batch gradient by that batch's actual number of examples.
In the report , create two graphs, for training and validation loss. It should have the loss on the y_axis and keep time in the x_axis to analyse the model improvement with time. In the report mention which mehtod reduces the loss the fastest and for each model when does the model start to overfit. Each graph must contain the plots for all the four models.

Submission instructions

Submit a single file `part_a.py` in the submission folder which you will submit on Moodle. Also add the relevant graphs and reasoning in **report.pdf** which you will keep in the submission folder. The evaluator will run it once per method:

```
python3 part_a.py part_ab_train.csv part_ab_test_public.csv full_batch \
    predictions_full_batch.txt weights_full_batch.txt

python3 part_a.py part_ab_train.csv part_ab_test_public.csv mini_batch \
    predictions_mini_batch.txt weights_mini_batch.txt

python3 part_a.py part_ab_train.csv part_ab_test_public.csv sgd \
    predictions_sgd.txt weights_sgd.txt

python3 part_a.py part_ab_train.csv part_ab_test_public.csv adagrad \
    predictions_adagrad.txt weights_adagrad.txt
```

`weights.txt` must contain exactly 79 lines: the first line is the bias b (3 comma-separated values, for classes N, A, O in that order), and each of the remaining 78 lines is one row of W (again 3 comma-separated values), in the same order as the feature columns in `part_ab_train.csv`.

`predictions.txt` must contain exactly one line per row of `part_ab_test_public.csv`, each line containing the three predicted class probabilities $p(N), p(A), p(O)$ (comma-separated, summing to 1 within numerical tolerance), in the same row order as the input file. Do not submit hard class labels.
For self-verification before submitting, reference weight snapshots after each of the first **5** training epochs, for all four methods are added at the link.

Evaluation

Your predictions are scored by Macro-AP on a held-out label set. Additionally, the evaluator checks that your weights match numerically to our implementation.

Part (b): Class Imbalance 15 points

The three classes in the current dataset are far from balanced: Normal rhythm is the majority class, Other rhythm a substantial minority, and AF the smallest class by a wide margin. In this part we will try to deal with Class Imbalance.

Required methods

We will fix the model to **mini-batch AdaGrad** with the hyperparameters $T_{\text{ada}}, \eta_{\text{ada}}, B_{\text{ada}}, \epsilon$ from Part (a) , to isolate the effect of the imbalance from the choice of optimiser.

Method 1: Baseline. Train on the original, untreated training set. No special handling.

Method 2: Inverse-frequency class weighting. Let n_k be the number of training examples of class $k \in \{0, 1, 2\}$ and $n = n_0 + n_1 + n_2$. Compute

$$\alpha_{y_i} = \frac{n}{3\, n_{y_i}}.$$

Train on all original examples with the weighted loss

$$\mathcal{L}(W, b) = \frac{\sum_{i=1}^{n} \alpha_{y_i} \left(-\log p_{y_i}\right)}{\sum_{i=1}^{n} \alpha_{y_i}},$$

RG: It may be easier for students if you could stick to the notations used in the class.
where p_{y_i} is the predicted probability of example i's true class. When training with mini-batches, apply this same normalisation *within each batch* (weights in the batch summing to 1) rather than globally.

$$g_W = X_B^\top \left[(P_B - Y_B) \odot \frac{\alpha_B}{\sum_{i \in B} \alpha_i} \right], \qquad g_b = \mathbf{1}^\top \left[(P_B - Y_B) \odot \frac{\alpha_B}{\sum_{i \in B} \alpha_i} \right].$$

For the untreated Baseline model this reduces exactly to Part (a)'s formula ($\alpha_i \equiv 1$ makes $\alpha_B / \sum_{i \in B} \alpha_i = \mathbf{1}/|B|$). This is the **full-strength** weighted model.

Method 3: Class weighting2 Using the same $\alpha_{y_i} = n/(3\, n_{y_i})$ values as Method 2, train a **second** weighted model (`classweight2`) with $\alpha_{y_i}^{p}$, $p = 0.3$ (fixed), in place of α_{y_i} , otherwise identical to Method 2.

Method 4: Focal loss. Replace cross-entropy with multiclass focal loss:

$$\mathcal{L}(W, b) = -\frac{1}{n} \sum_{i=1}^{n} \alpha'_{y_i} (1 - p_{i,y_i})^{\gamma} \log p_{i,y_i}$$

where p_{i,y_i} is the predicted probability of the true class. Use

$$\gamma = 2.0, \qquad \alpha'_c = (\alpha_c)^{0.5}.$$

Unlike fixed class weighting, the factor $(1 - p_{i,y_i})^{\gamma}$ is recomputed from the model's current prediction at every update.

Use the same mini-batch AdaGrad protocol as Part (a). For example i, let $t = y_i$ and z_k be the pre-softmax logit. The focal-loss gradient is

$$\boxed{\frac{\partial \mathcal{L}_i}{\partial z_k} = \alpha'_t (1-p_t)^{\gamma-1} \left[(1-p_t) - \gamma p_t \log p_t\right] (p_k - \mathbb{1}[k=t])}.$$

Average these gradients over the mini-batch; do not regularize anywhere.
For numerical stability, clip p_t before evaluating $\log p_t$, $p_t \in [10^{-12}, 1-10^{-12}]$. As a self-check, $\gamma = 0$ and $\alpha'_c = 1$ must reduce exactly to ordinary softmax cross-entropy.
You will therefore fit **four models**: Baseline, full-strength classweighting, classweighting2, and focal loss.
For self-verification before submitting, reference weight snapshots after each of the first **5** training epochs, for all four methods are added at the link.

Submission instructions

Submit a single file `part_b.py`. The evaluator will run it once per method:

```
python3 part_b.py part_ab_train.csv part_ab_test_public.csv baseline \
    predictions_baseline.txt weights_baseline.txt

python3 part_b.py part_ab_train.csv part_ab_test_public.csv classweight \
    predictions_classweight.txt weights_classweight.txt

python3 part_b.py part_ab_train.csv part_ab_test_public.csv classweight2 \
    predictions_classweight2.txt weights_classweight2.txt

python3 part_b.py part_ab_train.csv part_ab_test_public.csv focal \
    predictions_focal.txt weights_focal.txt
```

Output file formats are identical to Part (a).

Evaluation

Each method is scored separately (Macro-AP and Balanced Accuracy both reported) on held-out labels, and your weights are checked against a reference implementation as in Part (a).

Part (c): Feature Engineering 15 points

This part will be graded competitively. You can use AI tools and open source code as long as you disclose it in the report.

In Parts (a) and (b) you used a fixed, given 78-dimensional feature representation. In this part the data is derived from a secret binary AF dataset. On top of those 78 features, you will additionally be given:

- 314 further precomputed feature columns, obtained by running the published feature-extraction code of Goodfellow et al. over the same windows. Reference code is at this link.
- the underlying **raw, filtered ECG signal** itself for each 30-second window,

$$x_{\text{raw}} \in \mathbb{R}^{7500} \qquad (250 \text{ Hz} \times 30 \text{ s}),$$

You may use any subset of the 392 given feature columns, engineer further features on top of them, and/or work from x_{raw} directly your feature map $\phi(x)$ must have output dimension $d < 500$. Your final classifier must remain (binary) logistic regression,

$$p(y=1 \mid x) = \sigma\big(w^\top \phi(x) + b\big), \qquad \sigma(z) = \frac{1}{1+e^{-z}},$$

scoring AF (class 1) against non-AF (class 0).

Data

The data for this part is released as a private Kaggle Dataset attached to the competition , it is not publicly listed on Kaggle and only becomes visible once you join the Kaggle competition.
The released folder contains:

- `train.csv`, `val.csv` : columns `release_id`, `patient`, `label` (0=non-AF, 1=AF), the 78 baseline feature columns from Parts (a)/(b), and the 314 Goodfellow feature columns described above (392 feature columns, 3 metadata columns in total).
- `test.csv` same columns as train.csv, without `label`.
- `raw_signals/{patient}.npz` : one file per patient (60 total), each holding two row-aligned arrays: `release_id` (shape $(n,)$) and `signal` (shape $(n, 7500)$, `float32`) : the raw, filtered 30-second ECG window each row's features were computed from. Find a given row's window with `np.where(npz['release_id'] == row_release_id)`.

Patient-wise split. The 60 patients are split 36/12/12 across train/val/test with no patient appearing in more than one split, chosen so all three splits have close to the same AF prevalence . `patient` identifies which patient a row came from; it is provided so you can do patient-grouped cross-validation (never split one patient's rows across folds) and must **not** be used as a predictive feature.
`release_id` is a de-identified hash key joining a feature row to (i) its row in `raw_signals/` and (ii) the `Id` column of the Kaggle leaderboard. It carries no predictive information and must **not** be used as a model input. There may be NaN's , please use imputation to handle them.

Evaluation

Let P be the number of true AF (class 1) windows in the held-out set, N the number of true non-AF windows, TP the AF windows your submission correctly labels AF, and FP the non-AF windows it incorrectly labels AF. You are scored by

$$M = \frac{\mathrm{TP}}{P} - \frac{\mathrm{FP}}{N\,\epsilon}, \qquad \epsilon = \frac{1}{100},$$

Because that penalty is large, a false-positive rate above roughly ϵ (1%) will dominate M regardless of your recall. Thus aim for good recall while keeping False positives to almost none. A submission with $TPR < 0.1$ will not be eligible for grading.

Submission format

Submit a single file `part_c.py`, run as

```
python3 part_c.py dataset_dir/ model.pkl final_features.csv
```

where `dataset_dir` is the folder described above. Your script must:

1. train your model using `train.csv` (and, if you like, `val.csv`) and pick a decision threshold;
2. write `model.pkl`, a pickled dictionary
   {'weights': <array, shape (d,)>, 'bias': <float>, 'threshold': <float>};
3. compute your *final preprocessed* feature representation $\phi(x)$ for every row of `test.csv` and write it to `final_features.csv` as columns `feat_0`, ..., `feat_{d-1}` and a `release_id` column. d must equal `len(weights)` exactly.

`final_features.csv` must be the input to your logistic layer: your submission is scored purely using : $\sigma(\phi(x) \cdot \text{weights} + \text{bias}) \geq \text{threshold}$. Thus no post-processing on these features is allowed.

Refer to evaluate_partc.py so you can see exactly how the score below is computed from your two output files. Any of the file/folder names should not be hardcoded. **Your submission must run on Kaggle within 40 mins.**

Permitted methods

You may use any deterministic or stochastic gradient-based optimisation method, learning-rate schedule, regularisation, class-weighting scheme, or resampling strategy for training. Please used a fixed random seed everywhere so that results are reproduceable. You may use NumPy, SciPy, and scikit-learn for signal processing and feature computation. Any preprocessing step that estimates statistics (scaling, imputation, etc.) must be fit on the training set only and applied unchanged to validation/test data. For any other library, query on Piazza.

Performance on this task depends not only on the choice of classifier, but also on the complete modelling pipeline. In particular, careful data handling and validation can be as important as using a more sophisticated model. You are therefore encouraged to explore aspects such as:

- **Feature selection and preprocessing.** Not all provided features are necessarily equally informative. Feature scaling, removal of redundant or noisy features, and suitable feature-selection strategies may substantially affect performance. You can also refer to original challenge and use ideas from top submissions.
- **Decision-threshold selection.** The default probability threshold of 0.5 need not be optimal for the evaluation metric used in this task. Once a model has been trained, its decision threshold can be treated as an additional parameter. A useful approach is to determine the threshold using pooled out-of-fold predictions from patient-grouped folds on `train.csv`, combined with direct predictions on `val.csv`.
- **Data cleaning and noisy patients.** Some patients or recordings may behave as systematic outliers because of noisy measurements, unusual feature distributions, or unreliable labels. Out-of-fold predictions can be useful for identifying patients that consistently contribute a disproportionate number of errors.

Leaderboard

As per students request the leaderboard for this part will be on Kaggle. Here is the link. Please create a verified Kaggle account for the competition. The data described above is attached to it as a private Kaggle Dataset, visible only to joined participants.

Your actual score comes from us re-running your submitted `part_c.py` against a private evaluation set in the same format, not from the Kaggle metric score. It is just for your reference.

To submit to Kaggle: run `part_c.py` to get `model.pkl` and `final_features.csv`, then generate your submission with the script at the link.

```
python3 partc_kaggle.py model.pkl final_features.csv submission.csv
```

Report

(i) **Features.** State which features you used (given 392 / raw-signal-engineered) and your preprocessing steps. Name your top 10 features and how you ranked them.

(ii) **Class imbalance.** State the technique used to handle it and improvement it showed on M , TP, FP , Recall , FPR and Average Precision.

(iii) **Threshold selection.** Explain how your threshold was chosen. Plot M vs. threshold on `val.csv`, marking your chosen value. Then, sweeping $1/\epsilon$ uniformly over $[10, 1000]$, plot (a) the best achievable M and (b) the threshold that achieves it, each as a function of $1/\epsilon$. Explain the trend.

(iv) **Final metrics of submitted model** Table: M, TP, FP, AF recall, AF false-positive rate on `val.csv` at your chosen threshold and at 0.5, vs. your Part (a) baseline.

Submission Instructions & Guidelines

What you have been given

- `part_ab_train.csv`, and `part_ab_val.csv` (labelled) with `release_id` and the 78 baseline features for Parts (a)/(b);
- `train.csv`, `val.csv` and `test.csv` (unlabelled) with `release_id`, 392 features, and the raw 7500-sample signal for Part (c);

Held-out test labels are not released.

What to submit

Submit one archive named `<entry_number1>_<entry_number2>.zip`, containing at its top level:

```
part_a.py
part_b.py
part_c.py
report.pdf
```

Do not include datasets, generated predictions/weights files, virtual environments, or cache directories. Your programs must run without downloading anything from the internet, and must use only relative/command-line-supplied paths.

Report

Submit `report.pdf`, at most **4 pages**, including figures and tables. It must include:

Part (a). Two plots of training loss vs. time for all four methods on the same axes as described in the part(a) statement. Comparison among gradient descent methods as mentioned.

Part (c). Describe your feature pipeline (preprocessing steps, features used and their motivation) along with the required analysis and 2 plots as asked in part(c).
Disclose any AI use and open source code use in the report.

Evaluation intervention penalty:
A penalty equal to **10% of the marks allocated to the corresponding part** will be applied for every change required to your submission during evaluation. Examples include:

- renaming a submitted source file;
- changing a hard-coded input or output path;
- modifying the order of command-line arguments;
- manually creating a missing output file;
- correcting an output-file format;
- editing submitted source code to make it execute;
- changing assumptions about the working directory; or
- removing files or dependencies that interfere with evaluation;
- incorrect submission naming

Multiple interventions will incur multiple 10% penalties. A program that cannot be executed through minor interventions may receive zero marks for that part.